

11

Transactions, MVCC, WALs, and Checkpoints

This chapter introduces you to transactions, a fundamental part of every enterprise-level database system. Transactions are a way for a database to manage multiple operations, making them as though they were a single atomic operation. PostgreSQL has very rich and standard-compliant transaction machinery that allows users to specifically define transaction properties, including nested transactions.

PostgreSQL relies heavily on transactions to keep data consistent across concurrent connections and parallel activities, and thanks to **Write-Ahead Logs (WALs)**, PostgreSQL does its best to keep the data safe and reliable. Moreover, PostgreSQL implements **Multi-Version Concurrency Control (MVCC)**, a way to maintain high concurrency among transactions.

The chapter can be split into two parts: the first part is more practical and provides concrete examples of what transactions are, how to use them, and how to understand MVCC. The second part is much more theoretical and explains how WALs work, and how they allow PostgreSQL to recover even from a crash.

In this chapter, you will learn about the following topics:

- Transaction properties
- Transaction isolation levels
- What MVCC is and how it works
- Savepoints

- Deadlocks
- How PostgreSQL handles persistency and consistency: WALs
- VACUUM

Technical requirements

In order to proceed, you need to know the following:

- How to issue SQL statements via `psql`
- How to connect to the cluster and a database
- How to check and modify the cluster configuration

The chapter examples are available in the book's code repository and can be run on the *stand-alone* Docker image that you can find in the book's GitHub repository: <https://github.com/PacktPublishing/Learn-PostgreSQL-Second-Edition>.

Introducing transactions

A transaction is an atomic unit of work that either succeeds or fails. Transactions are a key feature of any database system and are what allows a database to implement the properties: **Atomicity, Consistency, Isolation, and Durability (ACID)**. Altogether, the ACID properties mean that the database must be able to handle units of work whole (atomicity), store data in a permanent way (durability), without inter-mixed changes to the data (consistency), and in a way that concurrent actions are executed as if they were alone (isolation).

You can think of a transaction as a bunch of related statements that, ultimately, will either all succeed or fail. Transactions are everywhere in a database, and you will have already used them even if you did not realize it: function calls, single statements, and so on are executed in a (tiny) transaction block. In other words, every action you issue against the database is executed within a transaction, even if you did not ask for it explicitly. Thanks to this automatic wrapping of any isolated statements into a transaction, the database engine can ensure its data is always consistent and protected from corruption, and we will see later in this chapter how PostgreSQL guarantees this.

Sometimes, however, you don't want the database to have control over your statements; rather, you want to be able to define the boundaries of transactions yourself, and of course, the database allows you to do it. For this reason, we use *implicit transactions* to describe transactions that the database starts for you without you needing to ask, and *explicit transactions* for those that you ask the database to start.

Before we can examine both types of transactions and compare them, we need a little more background on transaction concepts.

First of all, any transaction is assigned a unique number, called the **transaction identifier**, or **xid** for short. The system automatically assigns an xid to newly created transactions – either implicit or explicit – and guarantees that no two transactions with the very same xid exist at the same time in the database.

The other main concept that we need to understand early in our transaction explanation is that PostgreSQL stores the xid that generates and/o modifies a certain tuple within the tuple itself. The reason will be clear when we see how PostgreSQL handles transaction concurrency, so, for the sake of this part, let's just assume that every tuple in every table is automatically labeled with the xid value of the transaction that created the tuple.

You can inspect what the current transaction is by means of the special function `txid_current()`. So for example, if you ask your system a couple of simple statements, such as the current time, you will see that every `SELECT` statement is executed as a different transaction:

```

forumbd=> SELECT current_time, txid_current();
      current_time      | txid_current
-----+-----
16:51:35.042584+01 |      4813
(1 row)

forumbd=> SELECT current_time, txid_current();
      current_time      | txid_current
-----+-----
16:52:23.028124+01 |      4814
(1 row)

```

As you can see from the preceding example, the system has assigned two different transaction identifiers, respectively 4813 and 4814, to every statement, confirming that those statements have executed in different implicit transactions. You will probably get different numbers on your system.

If you inspect the special hidden column `xmin` in a table, you can get information about what transaction created the tuples; take the following example:

```

forumbd=> SELECT xmin, * FROM categories;
 xmin | pk |      title      |      description

```

```

-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
561 | 1 | DATABASE | Database related discussions
561 | 2 | UNIX | Unix and Linux discussions
561 | 3 | PROGRAMMING LANGUAGES | All about programming languages
(3 rows)

```

As you can see, all the tuples in the preceding table have been created by the very same transaction, number 561.



PostgreSQL manages a few different hidden columns that you need to explicitly ask for when querying a table to be able to see them. In particular, every table has the `xmin`, `xmax`, `cmin`, and `cmax` hidden columns. Their use and aim will be explained later in this chapter.

Now that you know that every transaction is numbered and that such numbers are used to label tuples in every table, we can move forward and see the difference between implicit and explicit transactions.

Comparing implicit and explicit transactions

Implicit transactions are those that you don't ask for but that the system applies to your statements. In other words, it is PostgreSQL that decides when the transaction starts and when it ends (transaction boundaries), and the rule is simple: every single statement is executed in its own separate transaction.

In order to better understand this concept, let's insert a few records into a table:

```

forumdb=> INSERT INTO tags( tag ) VALUES( 'linux' );
INSERT 0 1
forumdb=> INSERT INTO tags( tag ) VALUES( 'BSD' );
INSERT 0 1
forumdb=> INSERT INTO tags( tag ) VALUES( 'Java' );
INSERT 0 1
forumdb=> INSERT INTO tags( tag ) VALUES( 'Perl' );
INSERT 0 1
forumdb=> INSERT INTO tags( tag ) VALUES( 'Raku' );
INSERT 0 1

```

And let's query what the data in the table is:

```

forumdb=> SELECT xmin, * FROM tags;
  xmin | pk | tag | parent
-----+-----+-----+-----
  4824 |  9 | linux |
  4825 | 10 | BSD   |
  4826 | 11 | Java  |
  4827 | 12 | Perl  |
  4828 | 13 | Raku  |
(5 rows)

```

As you can see, the `xmin` field has a different (self-incremented) value for every single tuple inserted, which means a new transaction identifier (`xid`) has been assigned to the tuple or, more precisely, to the statement that executed `INSERT`. This means that every single statement is executed in its own single-statement wrapping transaction.



The fact that you see instances of `xid` incremented by a single unit is because, on the machine used for the examples, there is no concurrency, that is, no other database activity is going on. However, you cannot make any predictions about what the next `xid` will be in a live system, with different concurrent connections and running statements.

What if we had inserted all the preceding tags in one shot, being sure that if only one of them could not be stored for any reason, all of them would disappear? To this aim, we could use *explicit transactions*. An explicit transaction is a group of statements with a well-established transaction boundary: you issue a `BEGIN` statement to mark the start of the transaction, and either `COMMIT` or `ROLLBACK` to end the transaction. If you issue `COMMIT`, the transaction is marked as successful; therefore, the modified data is stored permanently. On the other hand, if you issue `ROLLBACK`, the transaction is considered to have failed and all changes disappear.

Let's see this in practice – add another bunch of tags, but this time within a single explicit transaction:

```

forumdb=> BEGIN;
BEGIN
forumdb=*> INSERT INTO tags( tag ) VALUES( 'PHP' );
INSERT 0 1

```

```

forumdb=> INSERT INTO tags( tag ) VALUES( 'C#' );
INSERT 0 1
forumdb=> COMMIT;
COMMIT

```

The only difference with respect to the previous bunch of INSERT statements is the explicit usage of BEGIN and COMMIT; since the transaction has committed, the data must be stored in the table:

```

forumdb=> SELECT xmin, * FROM tags;
 xmin | pk | tag | parent
-----+---+-----+-----
 4824 |  9 | linux |
 4825 | 10 | BSD   |
 4826 | 11 | Java  |
 4827 | 12 | Perl  |
 4828 | 13 | Raku  |
 4829 | 14 | PHP   |
 4829 | 15 | C#    |
(7 rows)

```

As you can see, not only is the data stored as we expected, but also both the last rows have the very same transaction identifier, that is, 4829. This means that PostgreSQL has somehow merged the two different statements into a single one.



When you issue an explicit transaction, psql changes its prompt, adding an asterisk to remind you that you are in an open transaction (i.e., a transaction that is not finished yet). If the transaction is aborted, due to an error, the asterisk is changed into a bang character.

Let's see what happens if a transaction ends with a ROLLBACK statement – the final result will be that the changes must not be stored. As an example, modify the tag value of every tuple to full uppercase:

```

forumdb=> BEGIN;
BEGIN
forumdb=> UPDATE tags SET tag = upper( tag );
UPDATE 7
forumdb=> SELECT tag FROM tags;

```

```
    tag
-----
LINUX
BSD
JAVA
PERL
RAKU
PHP
C#
(7 rows)

forumbd=*> ROLLBACK;
ROLLBACK
forumbd=> SELECT tag FROM tags;
    tag
-----
linux
BSD
Java
Perl
Raku
PHP
C#
(7 rows)
```

We first changed all the descriptions to uppercase, and the `SELECT` statement proves the database has done the job, but ultimately, we changed our mind and issued a `ROLLBACK` function. At this point, PostgreSQL throws away our changes and keeps the pre-transaction state.

Therefore, we can summarize that every single statement is always executed as an implicit transaction, while if you need more control over what you need to atomically change, you need to open (`BEGIN`) and close (`COMMIT` or `ROLLBACK`) an explicit transaction.

Being in control of an explicit transaction does not mean that you will always have a choice about how to terminate it; sometimes, PostgreSQL will not allow you to use `COMMIT` and consolidate a transaction because there are unrecoverable errors in it.

The most trivial example is when you input a syntax error:

```
forumdb=> BEGIN;
BEGIN
forumdb=*> UPDATE tags SET tag = uppr( tag );
ERROR:  function uppr(text) does not exist
LINE 1: UPDATE tags SET tag = uppr( tag );
                                ^
HINT:  No function matches the given name and argument types. You might
need to add explicit type casts.
Forumdb=!> COMMIT;
ROLLBACK
```

When PostgreSQL encounters an error, it aborts the current transaction. Aborting a transaction means that, while the transaction is still open, it will not honor any following command nor COMMIT and will automatically issue a ROLLBACK command as soon as you close the transaction. Therefore, even if you try to work after a mistake, PostgreSQL will refuse to accept your statements:

```
forumdb=> BEGIN;
BEGIN
forumdb=*> INSERT INTO tags( tag ) VALUES( 'C#' );
INSERT 0 1
forumdb=*> INSERT INTO tags( tag ) VALUES( PHP );
ERROR:  column "php" does not exist
LINE 1: INSERT INTO tags( tag ) VALUES( PHP );
forumdb=!> INSERT INTO tags( tag ) VALUES( 'Ocaml' );
ERROR:  current transaction is aborted, commands ignored until end of
transaction block
forumdb=!> COMMIT;
ROLLBACK
```

Anyway, handling syntax errors or misspelled object names are not the only problems you might find when running a transaction, and , anyway, those problems are quite simple to fix, but you might also find that your transaction cannot continue because there is some data constraint that prevents the statement from completing successfully. Imagine we don't allow any tags with a description shorter than two characters:

```
forumdb=> ALTER TABLE tags
          ADD CONSTRAINT constraint_tag_length
```

```
CHECK ( length( tag ) >= 2 );  
ALTER TABLE
```

Consider a unit of work that performs two different INSERT statements as follows:

```
forumdb=> BEGIN;  
BEGIN  
forumdb=*> INSERT INTO tags( tag ) VALUES( 'C' );  
ERROR: new row for relation "tags" violates check constraint "constraint_  
tag_length"  
DETAIL: Failing row contains (17, C, null).  
Forumdb=!> INSERT INTO tags( tag ) VALUES( 'C++' );  
ERROR: current transaction is aborted, commands ignored until end of  
transaction block  
forumdb=!> COMMIT;  
ROLLBACK
```

As you have seen, as soon as a DML statement fails, PostgreSQL aborts the transaction and refuses to handle any other statement. The only way you have to clear the situation is by ending the explicit transaction, and no matter which way you end it (either COMMIT or ROLLBACK), PostgreSQL will throw away your changes, rolling back the current transaction. The very same logic applies to implicit transactions: when a statement fails (for any reason), PostgreSQL rolls back the implicit transaction that wraps such a statement, and the end result is that the data is not persisted at all.

In the preceding examples, we have always shown the COMMIT ending for a transaction, but it is clear that when you are in doubt about your data, changes you have made, or an unrecoverable error, you should issue ROLLBACK. We have shown COMMIT to make it clear that PostgreSQL will prevent erroneous work from successfully terminating.

So, when are you supposed to use an explicit transaction? Every time you have a workload that must either succeed or fail, you have to wrap it in an explicit transaction. In particular, when losing a part of the work could compromise the remaining data is a good time to use a transaction. As an example, imagine an online shopping application: you surely do not want to charge your client before you have updated their cart and checked the availability of the products in storage. On the other hand, as a client, I would not want to get a message saying that my order has been confirmed, only to discover that the payment has failed for some reason.

Therefore, since all the steps and actions have to be atomically performed (check the availability of the products, update the cart, take the payment, confirm the order), an explicit transaction is what we need to keep our data consistent.